

Browser Monitors

Series: SYNTH | **Notebook:** 2 of 6 | **Created:** December 2025 | **Last Updated:** 01/28/2026

Creating and Optimizing Browser-Based Synthetic Tests

This notebook covers browser monitors in Dynatrace, including single-URL monitors, browser clickpaths, and performance analysis using the latest Dynatrace platform capabilities.

Table of Contents

1. [Browser Monitor Types](#)
 2. [Single-URL Monitors](#)
 3. [Browser Clickpaths](#)
 4. [Performance Metrics](#)
 5. [Validation and Assertions](#)
 6. [Analyzing Browser Results](#)
-

Prerequisites

-  Access to a Dynatrace environment with Synthetic Monitoring
-  Completed SYNTH-01 Fundamentals
-  Web application URL to monitor

1. Browser Monitor Types

Single-URL Browser Monitor

Loads a single page and captures performance metrics:

Feature	Description
Execution	Full Chrome browser render
Metrics	W3C Navigation Timing, resource timing
Screenshots	Automatic capture on completion/failure
Validation	Content validation, element checks

Best For:

- Homepage availability
- Landing page performance
- Single page applications (SPA) initial load

Browser Clickpath Monitor

Multi-step user journey simulation:

Feature	Description
Steps	Multiple pages/actions in sequence
Interactions	Click, type, select, wait
State	Cookies maintained within execution (session NOT maintained between executions)
Content Validation	Validate text/elements exist on page

Note: Unlike RUM session properties, browser clickpaths do not support extracting data into variables for use across executions. Content validation allows you to verify expected text/elements exist, but you cannot capture dynamic values for reuse.

Best For:

- Login flows
- Checkout processes
- Form submissions
- Multi-page workflows

Configuration Path

Dynatrace menu → Synthetic → Create synthetic monitor → Create browser monitor

2. Single-URL Monitors

Creating a Single-URL Monitor

1. URL Configuration

- Enter the full URL (https://...)
- Set viewport size (desktop, tablet, mobile)
- Configure user agent string

2. Execution Settings

- Frequency: 5-60 minutes
- Locations: Select public or private
- Timeout: Maximum execution time

3. Validation Rules

- HTTP status code validation
- Content validation (text, regex)
- Element presence checks

Viewport Presets

Preset	Dimensions	Use Case
Desktop	1920x1080	Standard desktop
Laptop	1366x768	Common laptop
Tablet	768x1024	iPad portrait
Mobile	375x667	iPhone 8

```
// List all browser monitors
fetch dt.entity.synthetic_test
| filter isNotNull(browserMonitorSubtype)
| fields id, entity.name, isEnabled, browserMonitorSubtype
| sort entity.name asc
| limit 50
```

```
// Browser monitor execution results (last 24h)
fetch dt.synthetic.events, from: now() - 24h
| filter matchesValue(event_type, "*browser*")
| fields timestamp,
    monitor = synthetic_test_id,
    location = synthetic_location_id,
    availability = execution_success,
    response_time_ms = toDouble(total_duration)
| sort timestamp desc
| limit 100
```

3. Browser Clickpaths

Creating Clickpath Monitors

Option 1: Record with Browser Extension

1. Install Dynatrace Synthetic Recorder (Chrome extension)
2. Start recording session
3. Perform user journey in browser
4. Stop recording and export to Dynatrace

Option 2: Manual Script Creation

Define steps programmatically using the script editor.

Common Actions

Action	Description	Example
navigate	Go to URL	Navigate to login page
click	Click element	Click login button
type	Enter text	Type username
selectOption	Select dropdown	Select country
wait	Wait for condition	Wait for element visible
javascript	Execute JS	Custom validation

Element Selectors

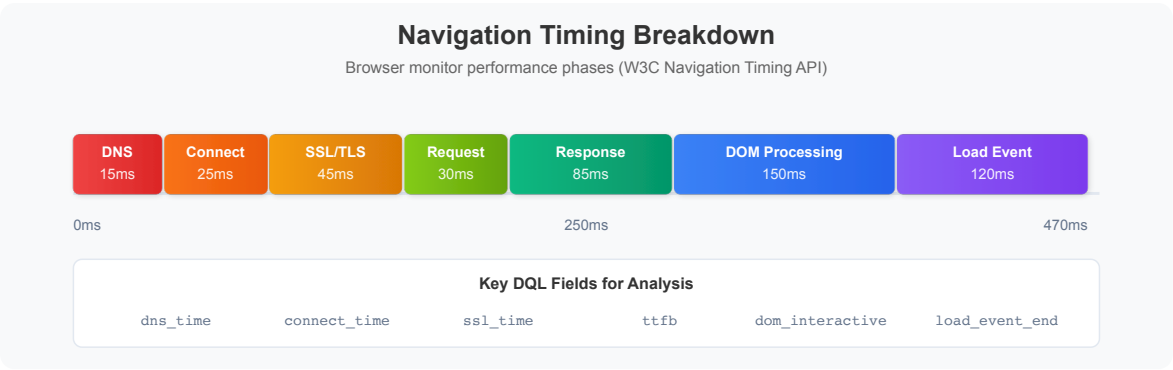
Selector Type	Example	Best Practice
CSS	#login-btn	Preferred - stable
XPath	//button[@id='login']	Complex structures
Link Text	Login	Simple links
Data Attribute	[data-testid='login']	Test automation

```
// Clickpath step performance analysis
fetch dt.synthetic.events, from: now() - 24h
| filter isNotNull(step_title)
| summarize {
    avg_duration_ms = avg(toDouble(step_duration)),
    max_duration_ms = max(toDouble(step_duration)),
    executions = count()
}, by: {synthetic_test_id, step_title}
| sort avg_duration_ms desc
| limit 30
```

4. Performance Metrics

W3C Navigation Timing

Browser monitors capture detailed timing metrics based on the W3C Navigation Timing API:



Key Performance Metrics

Metric	Description	Good	Needs Work
First Contentful Paint	First content rendered	< 1.8s	> 3.0s
Largest Contentful Paint	Largest element rendered	< 2.5s	> 4.0s
Time to Interactive	Page fully interactive	< 3.8s	> 7.3s
Total Blocking Time	Main thread blocked	< 200ms	> 600ms
Cumulative Layout Shift	Visual stability	< 0.1	> 0.25

Note: Targets based on Google's Core Web Vitals. Actual acceptable thresholds vary by application type and user expectations. Set baselines based on your own monitoring data.

```
// Browser monitor performance breakdown
fetch dt.synthetic.events, from: now() - 24h
| filter matchesValue(event_type, "*browser*")
| summarize {
    avg_dns_ms = avg(toDouble(dns_lookup_time)),
    avg_connect_ms = avg(toDouble(tcp_connect_time)),
    avg_ssl_ms = avg(toDouble(ssl_handshake_time)),
    avg_ttfb_ms = avg(toDouble(time_to_first_byte)),
    avg_dom_interactive_ms = avg(toDouble(dom_interactive_time)),
    avg_load_ms = avg(toDouble(total_duration)),
    executions = count()
}, by: {synthetic_test_id}
| sort avg_load_ms desc
| limit 20
```

```
// Performance trend over time
fetch dt.synthetic.events, from: now() - 7d
| filter matchesValue(event_type, "*browser*")
| makeTimeseries {
    avg_response_time = avg(toDouble(total_duration)),
    p95_response_time = percentile(toDouble(total_duration), 95)
}, interval: 1h
```

5. Validation and Assertions

Content Validation

Validation Type	Description	Example
Text Present	Page contains text	"Welcome"
Text Absent	Page doesn't contain	"Error"
Regex Match	Pattern matching	Order #\d{6}
Element Exists	DOM element present	#success-message
Element Content	Element has text	Button says "Submit"

HTTP Validation

Check	Default Behavior	Customization
Status Code	Fails on 4xx/5xx (400-599)	Can configure to ignore specific codes
Response Body	Content validation	Text/regex matching
Screenshots	Captured on success/failure	Automatic

Note: Response size validation and header validation are not available as built-in options. Use content validation or JavaScript steps for advanced checks.

Visual Validation

- Automatic screenshots on success/failure
- Visual comparison (pixel diff)
- Layout validation

```
// Failed browser monitor executions
fetch dt.synthetic.events, from: now() - 24h
| filter matchesValue(event_type, "*browser*")
| filter execution_success == false
| fields timestamp,
    monitor = synthetic_test_id,
    location = synthetic_location_id,
    error = error_message
| sort timestamp desc
| limit 50
```


6. Analyzing Browser Results

```
// Browser monitor availability by location
fetch dt.synthetic.events, from: now() - 24h
| filter matchesValue(event_type, "*browser*")
| summarize {
    total = count(),
    successful = countIf(execution_success == true),
    failed = countIf(execution_success == false)
}, by: {synthetic_test_id, synthetic_location_id}
| fieldsAdd availability_pct = round((successful * 100.0) / total, decimals:
2)
| sort availability_pct asc
| limit 30
```

```
// Response time distribution by location
fetch dt.synthetic.events, from: now() - 24h
| filter matchesValue(event_type, "*browser*")
| filter execution_success == true
| summarize {
    min_ms = min(toDouble(total_duration)),
    avg_ms = avg(toDouble(total_duration)),
    p50_ms = percentile(toDouble(total_duration), 50),
    p95_ms = percentile(toDouble(total_duration), 95),
    max_ms = max(toDouble(total_duration)),
    executions = count()
}, by: {synthetic_location_id}
| sort avg_ms desc
| limit 20
```

```
// Slowest page loads (outliers)
fetch dt.synthetic.events, from: now() - 24h
| filter matchesValue(event_type, "*browser*")
| filter execution_success == true
| filter toDouble(total_duration) > 5000 // > 5 seconds
| fields timestamp,
    monitor = synthetic_test_id,
    location = synthetic_location_id,
    response_time_ms = toDouble(total_duration)
| sort response_time_ms desc
| limit 20
```

Summary

In this notebook, you learned:

- ✓ **Browser monitor types** - Single-URL vs clickpath monitors
 - ✓ **Creating monitors** - URL configuration, viewports, locations
 - ✓ **Clickpath automation** - Recording, actions, selectors
 - ✓ **Performance metrics** - W3C timing, Core Web Vitals
 - ✓ **Validation** - Content, HTTP, and visual checks
 - ✓ **Analysis queries** - Availability, response times, failures
-

Next Steps

Continue to **SYNTH-03: HTTP Monitors** to learn about lightweight API monitoring.

References

- [Browser Monitors](#)
 - [Browser Clickpaths](#)
 - [Synthetic Recorder](#)
 - [Performance Metrics](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.